Master's thesis progres **Date:** October 8, 2026

1 Generic Domain Representation

The project adopts a generic and extensible instance representation to support the generation, diagnosis, and visualization of domain instances in Multi-Agent Path Finding (MAPF) and Flatland using the Clinguin framework. Instead of developing separate user interfaces for each application domain, this approach represents domain instances using five generic ASP predicates: object/2, node/1, property/3, position/3, and image/3. Each predicate represents a particular aspect of a domain instance, including objects, locations, object properties, object positions, and visual representations. Together, these predicates provide a common representation that separates domain-specific information from the generic user interface. This approach improves the flexibility, reusability, and extensibility of the system. In particular, new object types, properties, and visualization requirements can be introduced without redesigning the core representation or implementing a separate user interface for each domain. Below, I describe how I define and interpret each predicate.

1.1 Object declaration: object/2

What object exist, and what kind of object is it?

```
object(ObjectName, ObjectType).
```

For example:

```
object(track(1025),track).
object(car100,car).
object(robot10,robot).
object(agent(1),agent).
```

1.2 Node declaration: node/1

Answers what location exists.

```
node((Row,Column)).
```

```
node((R,C)) :- R=0..Rmax, C=0..Cmax.
```

1.3 Position relation: position/3

Answers Which object is associated with which location,and in what role?:

```
position(Node, Role, ObjectName).
```

It associates an object with a node under an arbitrary role.

The three arguments have the following meanings:

- `Node` represents a location declared through the generic predicate `node/1`.
- `Role` describes how the object is associated with the corresponding node.
- `ObjectName` identifies an object declared through the generic predicate `object/2`.

For example:

```
position((0,0),start,agent(1)).
position((10,10),end,agent(1)).

position((0,7),start,train(1)).
position((0,10),cell,track(1025)).
```

These facts state that:

- `agent(1)` is associated with node `(0,0)` under the role `start`;
- `agent(1)` is associated with node `(10,0)` under the role `end`;
- `train(1)` is associated with node `(0,7)` under the role `start`;
- `track(1025)` is associated with node `(0,10)` under the role `cell`.

The roles `start` and `cell` are examples defined by the corresponding domains. The generic UI does not assign a fixed semantic interpretation to these roles. Instead, it treats the `Role` argument as an uninterpreted ASP term whose meaning is determined by the loaded domain.

Consequently, different domains can introduce their own position roles without requiring changes to the generic representation. For example:

```
position((2,4),occupied,machine(2)).
```

This shows that the predicate is extensible: for planning and pathfinding future implementation that require temporal information, the model could even be extended with a position/4 predicate whose fourth argument represents a time step.

1.4 Properties: property/3

Answer What attribute does an object have, and what is its value?

```
property(ObjectName, PropertyName, Value).
```

Typical Flatland-oriented examples :

```
property(train(0),dep,0).
property(train(0),arr,12).
property(train(0),speed,1).
property(train(0),direction,2).
property(track(1025),code,1025).
```

1.5 Role-specific images: image/3

Answer Which image should represent this object when it is used in this role?

```
image(ObjectNama, Role, ImagePath).
```

examples :

```
image(track1025,cell,img/track1025.png).
image(track32800,cell,img/track32800.png).
image(track1024,cell,img/track1024.png)..
```

2 Active Domain

The generic UI is intended to visualize and manage different application domains, including Flatland, multi-agent path finding (MAPF), automated warehouses, and robotic environments. A user should be able to identify the domain associated with the UI without having to inspect the underlying files. For this reason, the interface introduces `active_domain/1`, whose single argument denotes a domain identifier. For example:

```
active_domain(flatland).
active_domain(MAPF).
active_domain(automatedwarehouse).
```

2.1 Domain identity from loaded files

A domain-specific file may declare `active_domain(flatland).` The generic interface consumes the declaration and displays the domain name in the top bar.

2.2 Missing declarations and interactive domain creation

When no domain-specific file is loaded, or when loaded files contain no `active_domain/1` fact, the UI it should display an empty-state message and an input for creating a domain identifier (for example, `warehouse`).

```
% Illustrative button action; the actual widget ID
% depends on the active-domain form implementation.
when(save(active_domain),click,call,
     add_atom(active_domain(
       _context_value(new_domain_name)))).
```

Unlike a file-provided fact, a session-created domain should expose *Edit* and *Remove* controls.

2.3 ASP rules for domain detection and editability

The following rules correspond to the proposed domain-state encoding and distinguish Clinguin-added atoms from ordinary loaded facts:

```
% File facts, if loaded, may directly assert
% active_domain(flatland).
```

```
user_added_domain(D) :-
    _clinguin_atom(active_domain(D)).

active_domain(D) :- user_added_domain(D).

has_active_domain :- active_domain(_).
no_active_domain :- not has_active_domain.

editable_domain(D) :-
    active_domain(D), user_added_domain(D).

readonly_domain(D) :-
    active_domain(D), not user_added_domain(D).
```

The rules express three core cases. (i) If there is no loaded or dynamic active-domain fact, `no_active_domain` holds and the UI may display the creation form. (ii) If a file asserts `active_domain(flatland)` with no corresponding dynamic atom, `readonly_domain(flatland)` holds. (iii) If the UI adds `active_domain(warehouse)`, `editable_domain(warehouse)` holds.

2.4 Presentation rules

A minimal declarative UI gate could be written as:

```
elem(active_domain_bar,container,w) :-
    has_active_domain.

elem(active_domain_missing,label,w) :-
    no_active_domain.
attr(active_domain_missing,label,
     "No active domain. Create a domain to begin.") :-
    no_active_domain.
```

The intended behavior is summarized in Table 1.

Table 1: Expected UI behavior according to the source of `active_domain/1`.

Domain state	UI presentation	Editing policy
No declaration	Empty state + create form	User may create
File-provided declaration	Name shown in top bar	Read-only
UI-inserted declaration	Name shown in top bar	Edit/remove dynamic atom

.

3 Generic Object Representation

An object is modeled as a binary predicate:

```
object(ObjectName,ObjectType).
```

`ObjectName` identifies a particular instance; `ObjectType` groups it under a category. Both are ASP terms. For instance:

```
object(train01,train).
object(track01,track).
object(station01,station).
```

The first example describes an object identified by `train01` that is categorized as a train.

3.1 Object identifiers: syntactic validity versus semantic meaning

The object-name field is intentionally flexible. Consider:

```
object(track01,train).
```

This is a syntactically valid ASP fact. Although the identifier `track01` sounds like a track to a human reader, `clingo` does not infer from that spelling that its type should be `track`. Therefore, the mismatch is *not logically inconsistent by itself.*

3.2 code: capturing the identifier

Add Object modal implements a text input and stores what the user enters in Clinguin context:

```
elem(input(add_object,new_object_name),textfield,
     add_object_name_field).
attr(input(add_object,new_object_name),
     placeholder,"e.g. train01").
when(input(add_object,new_object_name),input,context,
     (new_object_name,_value)).
```

4 Object Types: Two Sources of Input

The second argument of `object/2` comes from one of two *input pathways*: (i) types declared for the domain in loaded files, or (ii) types entered manually by the user. These are better described as *declared options and manual input.*

4.1 Path A: declared types from the domain description

A domain file may include:

```
active_domain(flatland).
domain_type(flatland,track).
domain_type(flatland,train).
```

The generic predicate `domain_type(D,T)` lists object categories made available by a specific domain. When these facts exist, the creation modal displays a dropdown so the user can select an existing type. Selecting an existing value is convenient, minimizes typing errors, and supports consistency across an instance.

The implementation is:

```
elem(input(add_object,declared_object_type),
     dropdown_menu,declared_type_section) :-
     has_declared_object_type.
elem(object_type_option(T),dropdown_menu_item,
     input(add_object,declared_object_type)) :-
     domain_type(_,T).
```

```
when(object_type_option(T),click,context,
     (new_object_type,T)) :- domain_type(_,T).
```

This is how *existing code* populates the dropdown, using the declared `domain_type/2` facts.

```
% Suggested refinement, not the current source rule:
elem(object_type_option(D,T),dropdown_menu_item,
     input(add_object,declared_object_type)) :-
     active_domain(D), domain_type(D,T).
```

4.2 Path B: manually entering a new or undeclared type

Suppose the loaded Flatland description lists only `track` and `train`, but the user wants:

```
object(station01,station).
```

A strict dropdown-only implementation would reject this workflow or leave it impossible to express. Instead, the UI offers a separate manual-type mode. The user switches to *Enter manually*, types `station`, and saves the object. The existing implementation uses:

```
elem(use_manual_type,button,object_type_mode_actions) :-
     has_declared_object_type.
when(use_manual_type,click,update,
     (declared_type_section,visibility,hidden)) :-
     has_declared_object_type.
when(use_manual_type,click,update,
     (manual_type_section,visibility,shown)) :-
     has_declared_object_type.

elem(input(add_object,manual_object_type),textfield,
     manual_type_section).
when(input(add_object,manual_object_type),input,context,
     (new_object_type,_value)).
```

Both declared-type selection and manual entry assign a value to the same context key, `new_object_type`; this allows the creation operation to stay independent of the chosen input mode.

4.3 Path C no loaded domain-specific files

The manual pathway is also the fallback when the UI has no declared type facts. The code makes the manual section visible under this condition:

```
attr(manual_type_section,visibility,hidden) :-
     has_declared_object_type.
attr(manual_type_section,visibility,shown) :-
     not has_declared_object_type.
```

5 Committing an Object

The creation modal ultimately delegates atom insertion to the backend:

```
when(save(add_object),click,call,
     add_atom(object(
```

```
        _context_value(new_object_name),
        _context_value(new_object_type)))).
```

The context values are substituted when the action is executed, and the Clinguin backend's `add_atom` operation makes the resulting atom available to the domain state. The action is separate from the input events that merely update context. The corresponding end-to-end workflow is:

1. Open *Create object* and enter an identifier, for example `station01`.
2. Select an existing type, if suitable, or switch to manual entry and type `station`.
3. Activate *Create object*; the backend receives an atom-creation request.
4. The updated instance can contain `object(station01,station)`, which the generic catalogue can display and inspect.

6 Adding Nodes and Initializing the Grid Workspace

6.1 Generic node representation and coordinate convention

The predicate `node/1` declares a location. For a two-dimensional rectangular workspace we represent the sole argument as the tuple `(R,C)`, where R is the row and C is the column. If the height is h and the width is w, the valid zero-based coordinates satisfy

$$0 \le R < h, \qquad 0 \le C < w,$$

so an $h \times w$ grid contains $h\,w$ nodes. Thus, for a grid with ten rows and ten columns, the range extends from `(0,0)` to `(9,9)` and comprises 100 locations. The order matters: the first coordinate is the *row* (height direction), while the second is the *column* (width direction).

```
node((0,0)).
node((0,1)).
node((1,0)).
% ... additional declared grid cells
```

6.2 Domain files and position-derived nodes

A loaded domain file may directly assert `node((R,C))` facts. In addition, the first argument of `position/3` is a location where an object is associated with a role:

```
position((2,4),start,train1).
```

This fact reveals a *referenced location*; it does not logically entail `node((2,4))` in ASP without an explicit rule. we may include the following rule:

```
node(N) :- position(N,_,_).
```

6.3 Session-generated grid logic

The implementation stores the dimensions as Clinguin *context* values and uses these values to derive a set of nodes. The essential rules are:

```
ui_grid_dimensions(Rows,Cols) :-
    _clinguin_context(new_rows,Rows),
    _clinguin_context(new_cols,Cols),
    Rows > 0, Cols > 0.

ui_grid_node((R,C)) :-
    ui_grid_dimensions(Rows,Cols),
    R = 0..Rows-1, C = 0..Cols-1.

node((R,C)) :- ui_grid_node((R,C)).

has_ui_grid :- ui_grid_node(_).

domain_node((R,C)) :-
    node((R,C)), not ui_grid_node((R,C)).

has_domain_nodes :- domain_node(_).
```

```
elem(create_grid,button,catalogue_grid_actions) :-
    not node((_,_)).
when(create_grid,click,context,
     (dialog,form(grid_dimensions))) :-
    not node((_,_)).
when(create_grid,click,update,
     (dialog(form(grid_dimensions)),visibility,shown)) :-
    not node((_,_)).
```

The associated modal prompts for *Rows* and *Columns*; its text fields write the context keys `new_rows` and `new_cols`. These values drive the declarative grid-generation rules upon a domain-state/UI recomputation. This allows an instance to be initialized even when no domain-specific file is loadede.

```
when(input(grid_dimensions,new_rows),input,
     context,(new_rows,_value)).
when(input(grid_dimensions,new_cols),input,
     context,(new_cols,_value)).
```

6.4 Editing and removal policy

The grid header exposes *Edit* and *Remove* buttons only when `has_ui_grid` is true:

```
elem(edit_grid,button,catalogue_grid_actions) :-
    has_ui_grid.
elem(remove_grid,button,catalogue_grid_actions) :-
    has_ui_grid.
```

The edit action opens the existing dimensions modal, so modifying the values changes the entire generated coordinate set. The remove confirmation uses context updates to set both dimensions to zero:

```
when(confirm_remove_grid,click,context,
     (new_rows,0)) :- has_ui_grid.
when(confirm_remove_grid,click,context,
     (new_cols,0)) :- has_ui_grid.
```

The positivity guard in `ui_grid_dimensions/2` then fails, and consequently `ui_grid_node/1` derives no nodes. No `remove_atom(node(...))` operation is needed, and file-provided nodes are not removed.

7 Adding Images

How can a generic Clinguin interface associate role-dependent images with objects, including objects introduced interactively, when image rules or visual assets are absent from domain files? How does a custom backend enable users to upload local files and expose them through the generic `image/3` representation?

The generic interface separates an object's identity and attributes from its appearance. The image/3 predicate has the form `image(Object,Role,ImagePath)`: the first argument identifies an object; the second specifies the visualization role, such as `start`, `end`, or `cell`; and the third identifies a frontend-accessible image resource. The same object can have different images for different roles. The UI should accommodate automatically derived images, images specified in domain files, and images uploaded after an object has been created. This is essential for reusing a common interface across MAPF, Flatland, and unforeseen application domains.

7.1 Images specified or derived by domain-specific rules

For MAPF agents, simple type-dependent rules can provide visual defaults:

```
image(O,start,"img/agent.png") :- object(O,agent).
image(O,end,"img/rectangle.png") :- object(O,agent).
```

These rules produce role-dependent `image/3` atoms for each `object(O,agent)` in the relevant domain model. They are *derived* facts, not individual image files uploaded by a user. For Flatland track objects, images may instead depend on an encoded track property; for example, one mapping could be:

```
image(O,cell,"img/straight_0.png") :-
    object(O,track), property(O,code,32800).
```

This mapping is applicable only to track codes covered by the domain's image rules. It is not a general replacement for visual data about trains, stations, or newly introduced categories. If the UI creates `object(train01,train)` without an applicable image rule or fact, the system cannot infer an appropriate picture from the identifier or type alone.

7.2 Two approaches and the motivation for manual uploads

File-defined or derived images. The domain author can predefine `image/3` facts or rules that associate known objects and roles with resources under an asset directory. This is convenient when object types and their visual conventions are known in advance, but insufficient when a user creates a new object or needs an image absent from the available asset catalogue.

UI-uploaded images. The implemented alternative allows a user to select an image from their own computer, associate it with an object and role, and upload its bytes to the Clinguin server. The server saves the file and inserts a session-owned `image/3` atom. This permits the same generic visualization workflow to support newly created trains, stations, robots, warehouse

equipment, and other domain objects without editing the domain encodings.

7.3 Current Clinguin visual-inspector workflow

The UI implementation contains a Visuals section for the inspected object. It lists `image(O,R,I)` entries, displays image previews with their roles and paths, and distinguishes session-added images from domain/derived images. Its upload form offers either a declared role or a manually entered role; consequently, the absence of `type_role/2` declarations does not prevent a new role from being supplied. The following extracts show the actual workflow:

```
% Show each object's image and its role in the inspector.
elem(inspector_visual(O,T,R,I),container,
     inspector_visual_list(O,T)) :-
    inspected_object(O,T), image(O,R,I).

% File picker for the currently inspected object.
elem(add_visual_file(O,T),file_input,
     inspector_add_role(O,T)) :- inspected_object(O,T).
attr(add_visual_file(O,T),accept,
     ".png,.jpg,.jpeg,.webp") :- inspected_object(O,T).

% A manual role is also supported.
when(add_visual_manual_role(O,T),input,context,
     (new_image_role,_value)) :- inspected_object(O,T).

% Delegate file storage and atom insertion to Python.
when(upload_visual(O,T),click,call,
     upload_object_image(O,
       _context_value(new_image_role))) :- inspected_object(O,T).
```

```
when(image_delete(O,T,R,I),click,call,
     remove_atom(image(O,R,I))) :-
    inspected_object(O,T), session_image(O,R,I).
```

7.4 Custom Clinguin backend: file validation and image atom creation

I extended `ClingoBackend` with a Python class named `GenericUIBackend`. The Angular `file_input` provides the filename and Base64-encoded contents through context keys `filename` and `filecontent`. The backend checks that both exist, validates the file extension against PNG, JPG, JPEG, and WebP, decodes the Base64 input, enforces a 5 MiB decoded-size limit, and performs basic file-header checks. Representative excerpts from the actual backend are:

```
class GenericUIBackend(ClingoBackend):
    IMAGE_DIR = Path("img/")
    ALLOWED_EXTENSIONS = {
        ".png", ".jpg", ".jpeg", ".webp"
    }
    MAX_IMAGE_SIZE = 5 * 1024 * 1024

    def upload_object_image(self, obj, role):
        filename = next(
            (c.value for c in self._context
             if c.key == "filename"), None)
        content = next(
```

```
            (c.value for c in self._context
             if c.key == "filecontent"), None)
        if not filename:
            raise RuntimeError("No image file was selected.")
        if not content:
            raise RuntimeError("No uploaded image content was found.")
        # See the supplied implementation for all checks.
```

The method subsequently constructs a safe filename fragment for the object and role, writes the decoded bytes to a server-local `img/` path, formats that path as an ASP string using `json.dumps`, and creates the new session atom by calling `self.add_atom`:

```
object_term = str(obj).strip('"')
role_term = str(role).strip('"')
safe_object = re.sub(
    r"[^A-Za-z0-9_-]+", "_", object_term
).strip("_")
safe_role = re.sub(
    r"[^A-Za-z0-9_-]+", "_", role_term
).strip("_")
stored_name = f"{safe_object}_{safe_role}{extension}"
stored_path = self.IMAGE_DIR / stored_name
stored_path.write_bytes(image_bytes)
path_term = json.dumps(stored_path.as_posix())
image_atom = (
    f"image({object_term},{role_term},{path_term})"
)
self.add_atom(image_atom)
self._messages.append((
    "Image uploaded",
    f"Added visual for {object_term} "
    f"with role {role_term}.", "success"
))
```

7.5 example

Assume the user previously created `object(train01,train)` through the generic object form, and that no predefined image matches the desired role. The user opens the object inspector, chooses or manually enters `start`, selects a local PNG file, and clicks *Upload image*. After successful validation and saving, the backend may add the following atom .

```
object(train01,train).
image(train01,start,"img/train01_start.png").
```

8 Adding Object Properties

8.1 Motivation: extensible object metadata

The generic representation includes `property/3`, whose arguments identify the object, property name, and property value, respectively:

```
property(ObjectName,PropertyName,Value).
property(train01,speed,2).
property(track01,length,50).
```

```
property(track01,length,50*cm).
```

This representation is intentionally open-ended: a future version of an application might require track length...

8.2 Declarative schemas with `type_property/4`

A domain may optionally declare expected attributes using the four-place predicate:

```
type_property(ObjectType,PropertyName,ValueType,Requirement).
% Illustrative declarations, not required for generic editing:
type_property(train,speed,int,required).
type_property(train,direction,const,required).
type_property(train,description,str,optional).
type_property(track,length,int,optional).
```

`ObjectType` is the category of object, `PropertyName` identifies an attribute, `ValueType` declares the expected representation or conversion policy, and `Requirement` distinguishes `required` from `optional`.

8.3 Operating with or without a domain schema

When a loaded domain supplies `type_property/4`, the inspector can show the expected attributes, identify missing required fields, and guide the user to complete the instance. If the domain is absent or omits type-property declarations, the `property/3` interface remains usable: the user types a property key and a value directly. Even with a complete schema, an additional custom property can be added. For example, a future Flatland instance can attach a length to a track even when the initial domain files did not define a track-length requirement.

8.4 Adding a previously undeclared custom property

The *Add new property* section of the provided inspector has independent fields for the property key and value. Neither form nor submit event is conditioned on the existence of `type_property/4`.

```
when(add_new_property_key(O,T),input,context,
    (new_custom_property_key,_value)) :- inspected_object(O,T).
when(add_new_property_value(O,T),input,context,
    (new_custom_property_value,_value)) :- inspected_object(O,T).
when(add_new_property_button(O,T),click,call,
    add_atom(property(O,
        _context_value(new_custom_property_key,const),
        _context_value(new_custom_property_value)))) :-
    inspected_object(O,T).
```

Thus, the generic inspector can be used for `property(track01,length,50)` even if only a `track` object exists and no track-specific properties were declared.

8.5 Editing and deleting user-created properties

The code restricts Edit/Delete buttons to properties present among Clinguin's session-added atoms. It preserves source-file facts by not offering controls for them:

```
elem(property_edit(O,T,P,V),button,
     configured_property_row(O,T,P,V)) :-
    inspected_object(O,T), property(O,P,V),
    _clinguin_atom(property(O,P,V)).
when(property_delete(O,T,P,V),click,call,
     remove_atom(property(O,P,V))) :-
    inspected_object(O,T), property(O,P,V),
    _clinguin_atom(property(O,P,V)).
when(property_edit_save(O,T,P,V),click,call,
     (remove_atom(property(O,P,V)),
      add_atom(property(O,P,
          _context_value(editing_property_new_value))))) :-
    inspected_object(O,T), property(O,P,V),
    _clinguin_atom(property(O,P,V)),
    _clinguin_context(editing_property_key,P),
    _clinguin_context(editing_property_old_value,V).
```

9 Challenge: Reconstructing the Original Flatland Representation from Generic ASP Predicates

A challenge encountered during the implementation of this project was developing an Answer Set Programming (ASP) encoding that reconstructs the original Flatland facts from the generic representation. The objective is to ensure that the reconstructed facts preserve the argument structure and identifier consistency expected by subsequent Flatland-specific processing stages.

The generic representation intentionally permits arbitrary object identifiers, making it flexible enough to represent different domains. However, this flexibility complicates the translation to Flatland, where the supplied instance uses numeric train identifiers consistently across several predicates. For example, the generic representation permits:

```
object(train01,train).
object(train(0),train).
object(my_train,train).
```

These declarations identify three distinct objects. In contrast, the original Flatland instance contains facts such as:

```
train(0).
start(0,(18,7),3,w).
end(0,(16,24),59).
speed(0,1).
```

Here, the first argument consistently identifies train `0`. A direct conversion based on the syntactic form of an object identifier is therefore insufficient:

```
train(ID) :- object(train(ID),train).
```

This rule works for `object(train(0),train)` but does not match `object(train01,train)` or `object(my_train,train)`.

Possible solution 1: Require structured object identifiers

One option is to require users to create Flatland objects using a prescribed identifier structure, such as `train(ID)` and `track(ID)`:

```
object(train(0),train).
object(track(1025),track).

train(ID) :- object(train(ID),train).
cell(N,Code) :- node(N), position(N,cell,track(Code)),
                object(track(Code),track).
```

This convention makes it possible to extract the train identifier and track code directly from the object term. However, it imposes domain-specific naming requirements on the generic UI and assumes that the identifier embedded in `track(Code)` is the actual transition code. Consequently, it reduces the flexibility that motivated the generic representation. If this option is adopted, the required identifier format should be enforced and validated whenever a Flatland object is created.

Possible solution 2: Store Flatland identifiers as object properties

A second option is to preserve unrestricted object names and store the Flatland-specific identifiers or codes as properties. For example:

```
object(train01,train).
property(train01,flatland_id,0).
property(train01,start_direction,3).
property(train01,start_status,w).
property(train01,end_value,59).
property(train01,speed,1).
position((18,7),start,train01).
position((16,24),end,train01).

object(track_a,track).
property(track_a,code,1025).
position((13,3),cell,track_a).
```

The corresponding conversion rules can then derive the original predicates without assuming a particular generic object-name structure:

```
train(ID) :- object(O,train), property(O,flatland_id,ID).

start(ID,N,D,W) :-
    object(O,train), property(O,flatland_id,ID),
    position(N,start,O), property(O,start_direction,D),
    property(O,start_status,W).

end(ID,N,V) :-
    object(O,train), property(O,flatland_id,ID),
    position(N,end,O), property(O,end_value,V).

speed(ID,S) :-
    object(O,train), property(O,flatland_id,ID),
    property(O,speed,S).

cell(N,C) :-
```

```
    node(N), object(O,track), position(N,cell,O),
    property(O,code,C).

cell(N,0) :- node(N), not position(N,cell,_).
```

This produces, among other facts, `train(0)`, `start(0,(18,7),3,w)`, `end(0,(16,24),59)`, `speed(0,1)`, and `cell((13,3),1025)`. The rule for empty cells assumes that a node without a track placement corresponds to a zero-coded Flatland cell.

For the property-based approach, additional validation is required to ensure that each train has exactly one numeric Flatland identifier, that different train objects do not share the same identifier, and that necessary attributes and placements exist.